

DIGITAL TWIN – AN AI-POWERED PERSONAL ASSISTANT

A Project Report

Submitted in fulfilment of the

Requirements for the award of degree of

Bachelor of Science of (Data Science)

By
Alka Lalit Dubey
Roll no. 9

Under the esteemed guidance of
Mr. Pragnesh Shah



DEPARTMENT OF DATA SCIENCE

**SHREE L. R. TIWARI DEGREE COLLEGE OF ARTS, COMMERCE
&
SCIENCE, Mira Road**

**(Affiliated to University of Mumbai) THANE,
401107 MAHARASHTRA**

2025-26 `

**SHREE L.R. TIWARI DEGREE COLLEGE OF ARTS, COMMERCE &
SCIENCE, MIRA ROAD**
(Affiliated to University of Mumbai)

DEPARTMENT OF DATA SCIENCE



CERTIFICATE

This is to certify this project report entitled “**DIGITAL TWIN – AN AI-POWERED PERSONAL ASSISTANT**”, submitted to University of Mumbai, Maharashtra is bonified record of work done by **Ms. Alka Lalit Dubey** for Academic Year 2024-2025 under the guidance of “**Mr. Pragnesh Shah**” from _____ to _____.

Date of Evaluation:

Internal Examiner

College Stamp

External Examiner

Head of the Department

Acknowledgement

I would like to convey my deep appreciation to my Guide **Mr. Pragnesh Shah** for his valuable guidance and encouragement in completion of my project work.

I would also like to express my gratitude to our principal **Dr. Sanjay Mishra** for providing all the required facilities to accomplish my work.

Alka Lalit Dubey

DECLARATION

I, **Alka Lalit Dubey (Roll No: 09)**, student of Shree L.R. Tiwari Degree College of Arts, Science and Commerce, Department of B.Sc. Data Science, hereby declare that this project report entitled “ **Digital Twin – An AI-Powered Personal Assistant**” submitted in the academic year 2025–26 is a record of my original work carried out under the guidance of **Mr. Pragnesh Shah**. This work has not been submitted for any other degree or diploma previously.

Alka Lalit Dubey

INDEX

Abstract

Chapter 1: Introduction

1.1 Background.....	1
1.2 Objectives	1
1.3 Purpose, Scope and Applicability.....	2
1.3.1 Purpose	2
1.3.2 Scope	2
1.3.3 Applicability	3
1.4 Achievements	3
1.5 Organization of Project.....	4

Chapter 2: Survey Technologies

2.1 Introduction... ..	5
2.2 Frontend Technologies.....	5
2.2.1 React.js	5
2.3 Backend Technologies.....	6
2.3.1 Flask.....	6
2.4 UI/UX Design Tools.....	6
2.4.1 Figma	6
2.4.2 Streamlit.....	7
2.5 Database Technologies	7
2.5.1 MySQL	7
2.6 Financial Analysis Tools	8
2.6.1 Python Libraries	8
2.7 Development Tools.....	9
2.7.1 Visual Studio Code (VSCode).....	9
2.8 Version Control System.....	9
2.8.1 Github	9

Chapter 3: Requirement and Analysis

3.1 Problem Definition	11
3.2 Requirement Specification	11
3.2.1 Functional Requirements	12
3.2.2 Non-Functional Requirements.....	12
3.3 Planning and Scheduling	12
3.3.1 Project Phases	13
3.4 Software and Hardware Requirements	13
3.4.1 Software Requirements.....	13
3.4.2 Hardware Requirements	13
3.5 Preliminary Product Description	14
3.6 Conceptual Models (Use Case Diagram)	14

Chapter 4: System Design

4.1 Basic Modules	17
4.2 Data Design	18
4.2.1 Schema Design	18
4.2.2 Data Integrity and Constraints.....	18
4.3 Procedural Design	19
4.3.1 Logic Diagrams	20
4.3.2 Data Structures	21
4.3.3 Algorithms Design.....	21
4.4.4 User Interface Design	21
4.5 Security Issues	22
4.6 System Ddesign	
Summary.....	23

ABSTRACT

In today's fast-paced world, individuals face increasing challenges in managing their time, tasks, and information efficiently. The need for a proactive system that not only organizes personal data but also predicts and reminds users of upcoming responsibilities has become critical. Digital Twin – An AI-Powered Personal Assistant is designed to address these challenges by combining artificial intelligence, machine learning, and natural language processing into a single intelligent system that supports productivity, decision-making, and personalized assistance.

The Digital Twin integrates seamlessly with user data to provide task prediction, proactive reminders, contextual question answering, and a personalized dashboard. Key features include a smart task manager, reminder system, and natural language query handling, all powered by a Flask backend, MySQL database, and an interactive web-based dashboard. By adapting to individual behavior and preferences, the assistant evolves continuously, ensuring accuracy, efficiency, and a user-friendly experience.

The project demonstrates the potential of extending digital twin technology beyond industrial and enterprise use cases into everyday personal productivity. This report outlines the project's objectives, scope, and technological foundation, followed by detailed chapters on tools and technologies, requirements and analysis, and system design. Together, these components present a comprehensive blueprint for building intelligent digital assistants capable of transforming the way individuals interact with their personal data

Chapter 1

Introduction

1.1 Background

In the modern era, individuals face increasing challenges in managing their time, tasks, and information effectively. Traditional productivity tools such as calendars, reminders, and task managers are often reactive and require constant manual input. The concept of a Digital Twin, originating from engineering and industrial domains, offers a new approach by creating a virtual replica of real-world entities.

When applied to personal productivity, a Digital Twin – An AI-Powered Personal Assistant can simulate user behavior, preferences, and daily routines. By leveraging Artificial Intelligence (AI), Machine Learning (ML), and Natural Language Processing (NLP), the system becomes proactive — predicting tasks, providing timely reminders, and answering contextual questions — ultimately enhancing personal efficiency and decision-making.

1.2 Objectives

The primary objectives of the Digital Twin project are:

- **To Develop a Digital Assistant:** Build a personal assistant that acts as a digital replica of the user, predicting upcoming tasks and offering intelligent reminders.
- **To Enable Contextual Interaction:** Integrate Natural Language Processing (NLP) so that users can ask questions in natural language and receive meaningful answers.
- **To Provide an Interactive Dashboard:** Design a user-friendly interface for managing tasks, reminders, and notes, making personal productivity seamless.
- **To Ensure Data Security:** Implement authentication and data protection mechanisms so that user data remains private and accessible only to authorized users.

- **To Demonstrate Digital Twin Applications:** Extend the concept of digital twins beyond industrial settings into personal productivity and individual decision-making.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The purpose of the Digital Twin project is to create an AI-powered personal assistant that helps individuals manage their schedules and information proactively. Unlike conventional tools, this system does not rely solely on user input but learns from patterns of behavior to predict and remind users about tasks, ensuring better time management and productivity.

- Provide proactive task reminders and intelligent predictions.
- Support contextual question answering using natural language.
- Deliver a simple yet effective dashboard for personal data management.
- Empower individuals to reduce stress and improve productivity through automation.

1.3.2 Scope

The scope of the Digital Twin encompasses:

- **Task Prediction & Reminders:** Using AI/ML models to predict upcoming tasks based on historical data.
- **Natural Language Support:** Allowing users to interact through simple queries.
- **Data Storage:** Using a secure MySQL database to manage user data such as tasks, reminders, and notes.
- **Dashboard Integration:** Offering a Flask-powered web dashboard with real-time interaction.

- **Scalability:** Providing a foundation for future integration with IoT devices, cloud services, and mobile applications.
-

1.3.3 Applicability

The Digital Twin is applicable to:

- **Students and Professionals:** Managing academic schedules, deadlines, meetings, and assignments.
- **Corporate Employees:** Handling work reports, meetings, and collaborative project tasks.
- **Personal Use:** Managing reminders, notes, and daily routines.
- **Research & Development:** Showcasing how digital twins can be applied beyond engineering into everyday personal assistance.

For example, a student preparing for exams can use the assistant to automatically schedule reminders for study sessions and deadlines, while a professional can rely on it to keep track of meetings and project deliverables.

1.4 Achievements

The development of the Digital Twin project marks several important milestones:

- **Flask Backend Integration:** A Flask-based backend successfully connected with a MySQL database for secure data management.
- **Task Prediction Models:** Implementation of machine learning algorithms to analyze user history and generate predictions.
- **Interactive Dashboard:** Development of a responsive web dashboard that allows users to manage tasks and reminders in real time.
- **NLP Integration:** Natural language support enabled for contextual query answering.
- **Secure Authentication:** Implementation of login features to ensure only authorized users can access their data.

1.5 Organization of Report

This report is structured to provide a systematic view of the project. The chapters are organized as follows:

- **Chapter 2:** Survey of Technologies – Provides an overview of the technologies used in the project including frontend, backend, database, UI/UX tools, and version control systems.
- **Chapter 3:** Requirements and Analysis – Explains the problem definition, requirement specifications, planning and scheduling, hardware/software requirements, and conceptual models.
- **Chapter 4:** System Design – Discusses the core modules, data design, logic flow, user interface, security considerations, and test cases.

Through this structure, the report provides a complete understanding of the project development, design process, and future implications

Chapter 2

Survey and Technologies

2.1 Introduction

The development of the **Digital Twin – An AI-Powered Personal Assistant** required the integration of several technologies to ensure efficiency, scalability, and a user-friendly experience. Since the system involves task prediction, proactive reminders, contextual question answering, and dashboard interaction, the chosen tools and frameworks had to support real-time performance, seamless integration, and flexibility.

This chapter provides a detailed survey of the technologies used in building the Digital Twin system, along with the rationale behind their selection.

2.2 Frontend Technologies

2.2.1 React.js

React.js is an open-source JavaScript library developed by Facebook for building modern web interfaces. It allows developers to create modular components and efficiently update only the required sections of the user interface using its Virtual DOM mechanism.

Advantages:

- **Component-Based Architecture:** Encourages modular and reusable design.
- **Virtual DOM:** Improves performance by updating only the affected components.
- **Cross-Platform Support:** Works across devices for responsive dashboards.
- **Community & Ecosystem:** Large library of extensions and support.
- **API Integration:** Easy to integrate with Flask APIs for real-time data.

Rationale for Choosing React.js:

React.js was chosen for its performance, modular design, and compatibility with modern tools, making it ideal for building the Digital Twin dashboard.

2.3 Backend Technologies

2.3.1 Flask

Flask is a lightweight Python web framework used to build web applications and APIs. It is known for its flexibility, simplicity, and ability to integrate with Python libraries.

Advantages:

- **Lightweight and Flexible:** Minimal structure with easy customization.
- **Seamless Python Integration:** Works directly with ML/NLP libraries.
- **Secure:** Provides authentication and session management.
- **Rapid Development:** Enables fast prototyping and testing.
- **REST API Support:** Allows smooth communication between frontend and backend.

Rationale for Choosing Flask:

Flask was chosen for its simplicity, scalability, and strong integration with AI/ML libraries, making it perfect for managing backend logic and APIs for the Digital Twin system.

2.4 UI/UX Design Tools

2.4.1 Figma

Figma is a cloud-based UI/UX design tool that supports collaborative interface design and prototyping.

Advantages:

- **Real-Time Collaboration:** Multiple designers can work together.

- **Cross-Platform:** Accessible via browser and devices.
- **Prototyping:** Allows interactive mockups for testing designs.
- **Consistency:** Ensures reusable design systems and templates.
- **Integration with React:** Smooth handoff to frontend developers.

Rationale for Choosing Figma:

Figma was chosen to design intuitive and user-friendly dashboard prototypes, ensuring a clean and professional interface for the Digital Twin project.

2.4.2 Streamlit

Streamlit is an open-source Python library that simplifies the creation of interactive dashboards and web applications directly from Python scripts.

Advantages:

- **Ease of Use:** Requires minimal code for building dashboards.
- **Direct Python Integration:** Works seamlessly with ML/NLP models.
- **Rapid Prototyping:** Useful for quick testing and experimentation.
- **Customizable Widgets:** Includes sliders, charts, and forms for interactivity.
- **Deployment-Friendly:** Can be deployed easily for demo purposes.

Rationale for Choosing Streamlit:

Streamlit was included as a backup UI/UX tool in case Figma integration becomes challenging. It provides a reliable way to quickly build interactive dashboards connected directly to Python-based machine learning models.

2.5 Database Technologies

2.5.1 MySQL

MySQL is a relational database management system (RDBMS) widely used for storing structured data.

Advantages:

- **ACID Compliance:** Guarantees data accuracy and reliability.
- **Scalability:** Handles growing datasets efficiently.
- **Performance:** Provides fast query execution.
- **Security:** Ensures user authentication and access control.
- **ORM Integration:** Works with Flask using SQLAlchemy ORM.

Rationale for Choosing MySQL:

MySQL was selected for its speed, reliability, and compatibility with Flask, making it the best choice for managing user tasks, reminders, and authentication data.

2.6 Machine Learning & NLP Tools

2.6.1 Python Libraries

The Digital Twin uses multiple Python libraries for task prediction, NLP, and data visualization:

- **Pandas:** For structured data manipulation.
- **NumPy:** For numerical computations.
- **Scikit-learn:** For predictive task modeling.
- **NLTK / SpaCy:** For natural language processing and query handling.
- **Matplotlib / Plotly:** For data visualization in dashboards.

Rationale for Choosing Python Libraries:

Python libraries were chosen for their rich ecosystem, flexibility, and seamless integration with Flask, making them essential for predictive analytics and contextual question answering.

2.7 Development Tools

2.7.1 Visual Studio Code (VS Code)

Visual Studio Code is a lightweight yet powerful IDE used by developers across technologies.

Advantages:

- **Extensible:** Supports plugins for Python, Flask, and React.
- **Integrated Terminal:** Helps in executing commands directly.
- **Git Support:** Simplifies version control management.
- **Debugging Tools:** Provides built-in debugging for multiple languages.

Rationale for Choosing VS Code:

VS Code was selected as the primary development tool because of its versatility, integration with GitHub, and support for all major frameworks used in this project.

2.8 Version Control System

2.8.1 GitHub

GitHub is a collaborative version control platform built on Git.

Advantages:

- **Version Tracking:** Keeps a history of changes.
- **Collaboration:** Supports multiple developers with pull requests.
- **Automation Support:** Enables continuous integration workflows.
- **Open-Source Ecosystem:** Access to a wide range of projects.
- **Documentation:** Provides wikis and README for clarity.

Rationale for Choosing GitHub:

GitHub was chosen to manage the codebase, ensure version control, and enable future collaboration, ensuring that project development remains structured.

Summary

This chapter surveyed the technologies employed in the Digital Twin project. The combined use of React.js (frontend), Flask (backend), Figma and Streamlit (UI/UX design), MySQL (database), Python libraries (ML/NLP), VS Code (development tool), and GitHub (version control) ensures a robust, scalable, and user-friendly system. This carefully selected stack creates a strong foundation for building a reliable AI-powered personal assistant.

Chapter 3

Requirements and Analysis

3.1 Problem Definition

In today's fast-paced lifestyle, individuals struggle with managing multiple tasks, remembering deadlines, and organizing information effectively. Traditional tools such as calendars, sticky notes, and reminder apps are helpful but limited because they require constant manual input and lack intelligence.

Key challenges faced by individuals include:

- **Task Overload:** Difficulty in tracking academic, professional, and personal commitments simultaneously.
- **Lack of Proactive Systems:** Most tools notify based only on manual entries, without predicting future needs.
- **Fragmented Information:** Users rely on multiple apps (notes, reminders, calendars), leading to inefficiency.
- **Time Management Issues:** Forgetting tasks and deadlines reduces productivity and creates stress.
- **Limited Contextual Assistance:** Existing systems cannot answer personalized queries using user data.

To overcome these challenges, there is a need for a Digital Twin – an intelligent AI-powered assistant that replicates user behavior, predicts tasks, reminds proactively, and answers contextual questions using personal data.

3.2 Requirement Specification

3.2.1 Functional Requirements

- **User Registration and Authentication:** Secure signup/login with password encryption.
- **Task Management:** Add, edit, delete, and organize tasks and reminders.

- **Predictive Task Modeling:** AI-based task prediction based on user history.
- **Proactive Reminders:** Automated notifications for upcoming tasks.
- **Natural Language Query Support:** Ability to answer contextual questions using NLP.
- **Notes Management:** Feature to store and organize notes.
- **Dashboard:** Interactive and user-friendly dashboard to view tasks, reminders, and notes.
- **Notification System:** Real-time alerts for tasks, reminders, and updates.
- **Admin Panel:** Allows administrators to manage users and monitor system activity.

3.2.2 Non-Functional Requirements

- **Scalability:** System should handle multiple users and data growth.
- **Security:** Strong authentication and data protection (password encryption, secure sessions).
- **Usability:** Intuitive UI/UX with easy navigation and accessibility.
- **Reliability:** Minimal downtime with consistent performance.
- **Performance:** Fast response to queries and smooth task management.
- **Portability:** Accessible on different devices (desktop, laptop, mobile browsers).

3.3 Planning and Scheduling

3.3.1 Project Phases

- **Phase 1:** Requirement Gathering and Analysis
- **Phase 2:** Design (UI/UX Prototyping in Figma/Streamlit and System Architecture)

- **Phase 3:** Development (Frontend in React.js, Backend in Flask, Database in MySQL)
- **Phase 4:** Integration of AI/ML modules (Task prediction, NLP)
- **Phase 5:** Testing (Functional, Non-functional, API testing with Postman)
- **Phase 6:** Deployment (on hosting platforms such as Heroku or AWS)
- **Phase 7:** Maintenance and Iterative Improvements

3.4 Software and Hardware Requirements

3.4.1 Software Requirements

- **Operating Systems:** Windows, macOS, or Linux.
- **Development Tools:**
 - **Frontend:** React.js, npm/yarn, Redux.
 - **Backend:** Python 3.x, Flask framework.
 - **Database:** MySQL.
 - **Design:** Figma (primary), Streamlit (backup).
 - **Libraries:** Pandas, NumPy, Scikit-learn, NLTK/SpaCy, Matplotlib, Plotly.
 - **Version Control:** Git and GitHub.
 - **Testing Tools:** Selenium, Postman for API testing.
 - **Deployment:** Heroku / AWS for hosting the application.

3.4.2 Hardware Requirements

- **Development Machines:**
 - **Processor:** Intel i5 or higher / Apple M1 chip.
 - **RAM:** Minimum 8 GB.
 - **Storage:** 256 GB SSD or higher.

- **Internet:** Stable broadband connection for collaboration and testing.

3.5 Preliminary Product Description

The Digital Twin is a personal productivity assistant designed to replicate user behavior and enhance efficiency. It provides a dashboard for managing tasks, reminders, and notes, while also offering AI-powered predictions and NLP-based question answering.

Key features include:

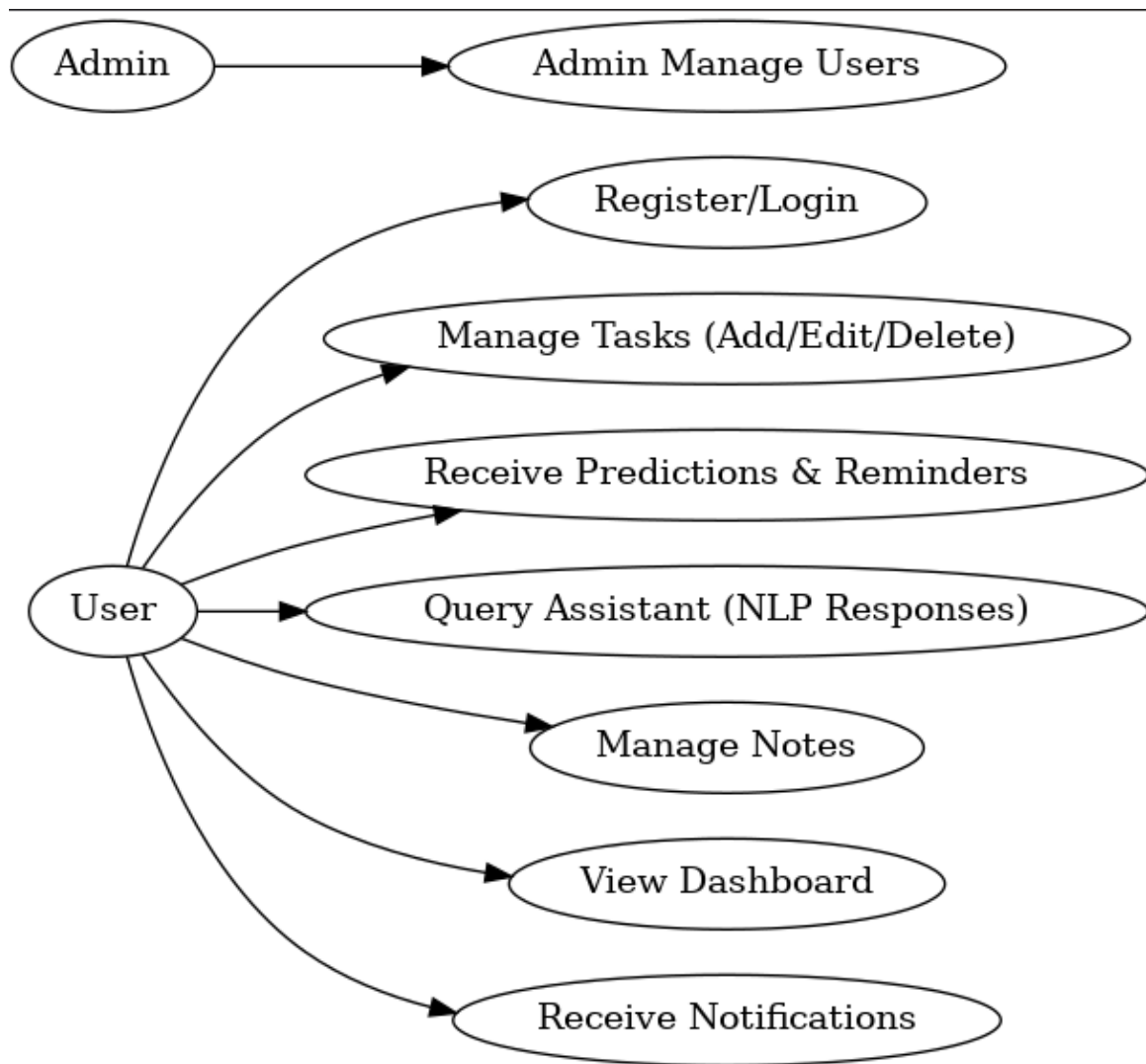
- **User-Friendly Interface:** Clean dashboard designed in Figma (with Streamlit as an alternative backup).
- **Task Prediction and Reminders:** Intelligent reminders generated from user history.
- **NLP Integration:** Responds to queries in natural language.
- **Notes Management:** Organize and store personal notes.
- **Data Security:** Secure login and encryption to protect sensitive data.

3.6 Conceptual Models

The use case diagram illustrates interactions between the user and the system.

Primary functions include:

- Register/Login
- Manage Tasks (Add/Edit/Delete)
- Receive Predictions & Reminders
- Query the Assistant (NLP-based responses)
- Manage Notes
- View Dashboard
- Receive Notifications
- Admin Manage Users



Chapter 4

System Design

4.1 Basic Modules

The **Digital Twin** system is designed as a collection of interconnected modules, each responsible for specific functionality. This modular approach ensures scalability, maintainability, and ease of testing.

- **Authentication Module**
Handles secure user registration, login, password encryption, and authentication tokens. Ensures only authorized users can access their data.
- **Profile Management Module**
Allows users to create and update their profiles, storing preferences and personal data for task prediction and personalization.
- **Task Management Module**
Enables users to add, edit, delete, and track tasks. Supports task categorization, priorities, deadlines, and completion status.
- **Prediction (AI/ML) Module**
Uses machine learning models to analyze user history and predict upcoming tasks. Suggests potential tasks and deadlines proactively.
- **Reminder & Notification Module**
Generates real-time reminders and notifications for tasks, ensuring users never miss deadlines. Supports email, in-app, and push notifications.
- **NLP / Query Handling Module**
Allows users to interact with the assistant using natural language queries. Provides contextual responses using NLP models and user data.
- **Notes & Context Store Module**
Stores personal notes and contextual information linked to tasks, enhancing NLP responses and predictions.
- **Dashboard / UI Module**
A user-friendly dashboard developed with React.js (primary) and Streamlit (backup). Displays tasks, reminders, predictions, and Q&A responses.

- **Admin Module**

Provides administrators with tools to manage users, monitor logs, and ensure smooth operation of the system.

4.2 Data Design

4.2.1 Schema Design

Table Name	Columns	Notes
User	user_id (PK), username, email (unique), password_hash, created_at, preferences	Stores user credentials & preferences
Task	task_id (PK), user_id (FK), title, description, due_date, priority, status, created_at	Each task is linked to a user
Reminder	reminder_id (PK), task_id (FK), user_id (FK), reminder_time, method, status	Reminders for tasks with method (email, notification, etc.)
Note	note_id (PK), user_id (FK), task_id (FK, nullable), content, created_at	Notes can be standalone or linked to a task
Prediction	prediction_id (PK), user_id (FK), task_title, confidence_score, generated_at	AI predictions about tasks with confidence level
QueryLog	query_id (PK), user_id (FK), query_text, response_text, timestamp	Stores logs of user queries and AI responses
AuditLog	log_id (PK), user_id (FK), action, ip_address, timestamp	Tracks user activities for security & monitoring

4.2.2 Data Integrity and Constraints

- **Primary Keys (PK):** Guarantee unique identification of records.

- **Foreign Keys (FK):** Enforce referential integrity between tables.
- **Unique Constraints:** Ensure uniqueness of critical fields such as email and username.
- **Check Constraints:** Validate field values (e.g., `due_date > created_at`, non-negative values for expenses).
- **Referential Integrity:** Cascade updates and deletions where required.
- **Indexing:** Used on frequently queried fields like `task_id`, `user_id`, and `due_date` for efficiency.

4.3 Procedural Design

A. User Registration Process

1. Input validation.
2. Check for existing email/username.
3. Hash password using bcrypt.
4. Store user details in database.
5. Send verification email.
6. Activate account upon confirmation.

B. Task Creation & Reminder Process

1. User enters task details.
2. Validate inputs (title, `due_date`, etc.).
3. Save task into database.
4. Prediction module updates user model.
5. Reminder module schedules notifications.
6. Dashboard updated in real-time.

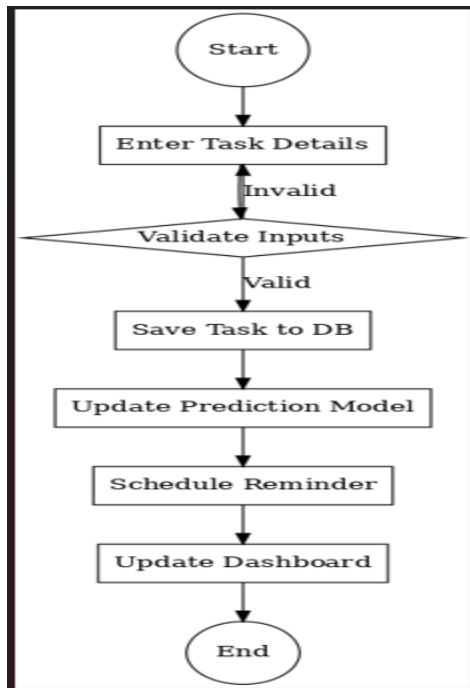
C. Prediction Workflow

1. Retrieve historical task data.
2. Apply ML preprocessing and feature engineering.
3. Generate predictions using classification/time-series model.
4. Save predictions in database.
5. Notify user if high-confidence prediction is found.

D. NLP Query Handling

1. User inputs natural language query.
2. NLP pipeline processes (tokenization, intent recognition).
3. Retrieve context from notes, tasks, or query log.
4. Generate response.
5. Display result on dashboard.

4.3.1 Logic Diagram



4.3.2 Data Structures

- **Lists / Queues:** Used for notification scheduling.
- **Dictionaries (HashMaps):** Store session data and preferences.
- **DataFrames (Pandas):** For ML preprocessing and analytics.
- **Objects/Classes:** Define User, Task, Prediction entities.

4.3.3 Algorithms Design

- **Task Prediction Algorithm:** Uses historical frequency and timestamps to predict future tasks. Scikit-learn classification models applied.
- **Reminder Scheduling Algorithm:** Implements priority-based scheduling using task deadlines.
- **NLP Query Algorithm:** Combines intent classification (spaCy/NLTK) with contextual retrieval (TF-IDF embeddings).
- **Sorting & Filtering Algorithm:** Efficient search using binary search trees or database indexing.

4.4 User Interface Design

The UI is designed in Figma (primary) with Streamlit as an alternative. It emphasizes simplicity, clarity, and responsiveness.

- **Color Scheme:** Professional and minimal (blue, white, grey) for productivity.
- **Typography:** Clean, readable fonts such as Roboto or Open Sans.
- **Navigation:**
 - **Header:** Logo, search bar, profile.
 - **Sidebar:** Quick access to Dashboard, Tasks, Notes, Predictions, Settings.

- **Forms:** Clear placeholders, validation, error messages.
- **Responsiveness:** Layout adapts to desktop, tablet, and mobile.
- **Accessibility:** WCAG-compliant with keyboard shortcuts and high-contrast colors.

4.5 Security Issues

The Digital Twin system incorporates multiple layers of security to ensure user data protection.

- **Authentication Security:**
 - Password hashing (bcrypt).
 - Multi-factor authentication (optional).
- **Authorization:**
 - Role-based access control (RBAC).
- **Data Encryption:**
 - SSL/TLS for in-transit data.
 - Database encryption for sensitive data at rest.
- **Input Validation:**
 - Sanitization to prevent SQL injection and XSS.
- **Session Management:**
 - HttpOnly, Secure cookies.
 - Session expiration and refresh tokens.
- **Audit Logging:**
 - Track critical actions for monitoring.
- **Regular Security Assessments:**
 - Penetration testing and vulnerability scans.
- **Compliance:**
 - Adheres to data protection regulations such as GDPR.

4.6 System Design Summary

The system integrates React.js/Streamlit frontend, Flask backend, MySQL database, ML/NLP modules, and notification services into a modular and scalable architecture. Each component communicates securely, ensuring reliability and performance.